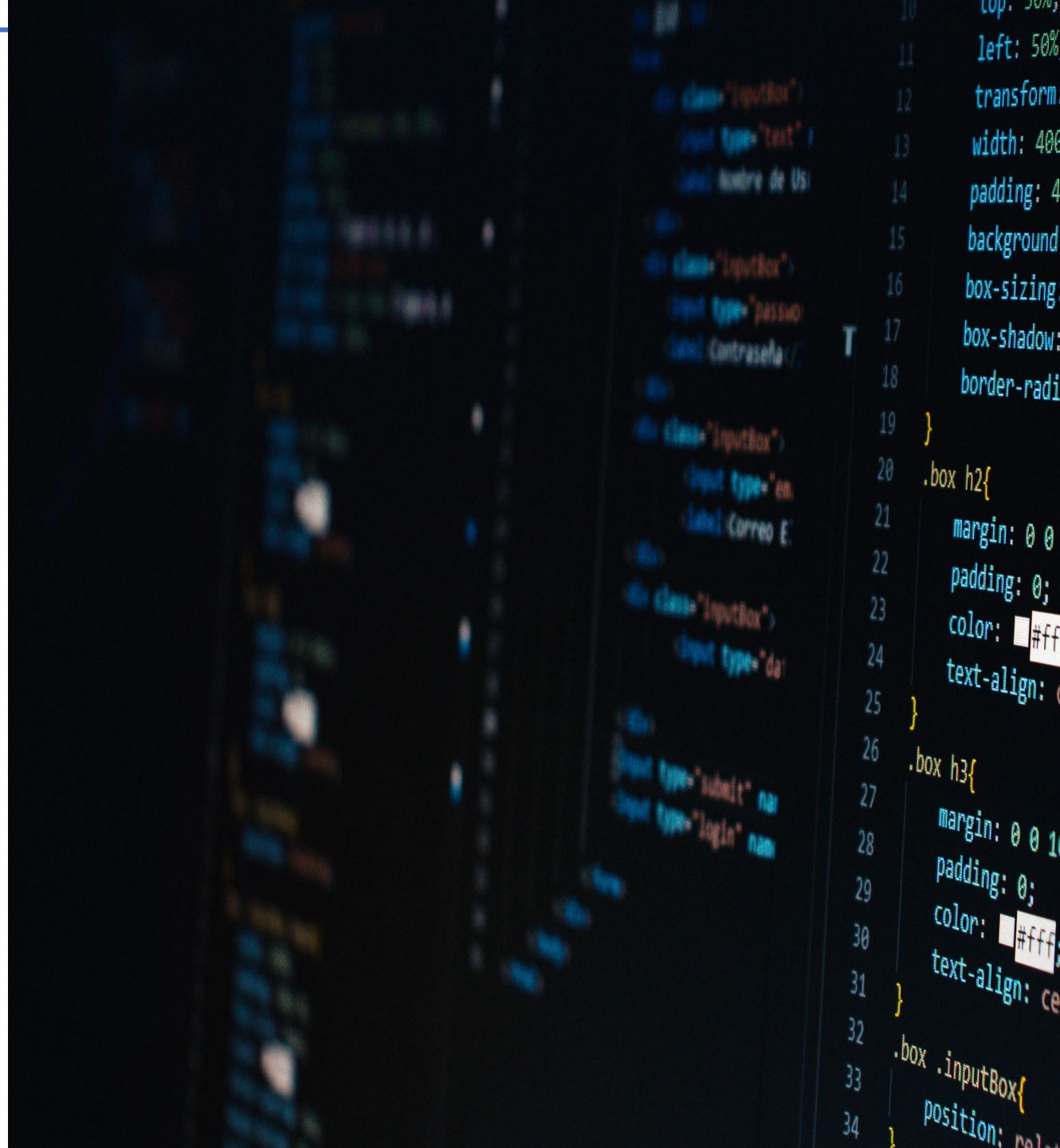
Bahan Ajar PBO

Bab 4

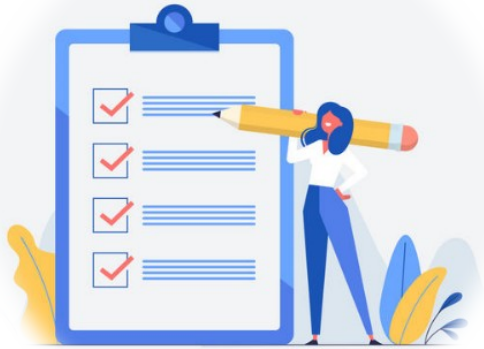
Kelas dan Objek

Oleh:
Abidatul Izzah, M.Kom
Fikha Rizky Aullia, M.Pd



DIII Manajemen Informatika, Kampus Kediri
POLITEKNIK NEGERI MALANG

Capaian Pembelajaran



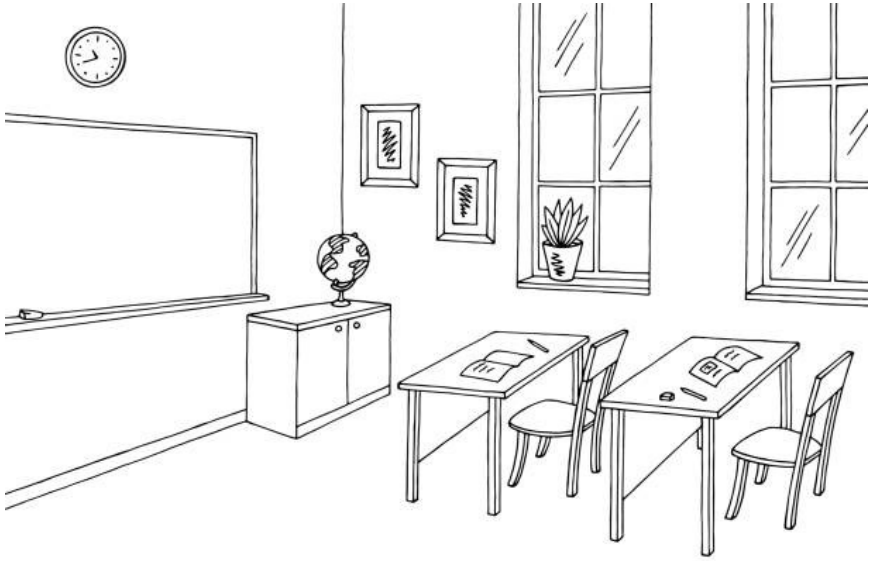
Setelah menempuh bab ini, diharapkan mahasiswa mampu menguasai kompetensi berikut:

1. Mampu menjelaskan kelas dan objek pada python
2. Mampu mengimplementasikan konsep kelas dan objek pada sebuah kasus program sederhana

Mengenal Kelas dan Objek

Kelas memiliki arti sebuah wadah yang dapat menampung seluruh property.

Dalam pemrograman, **kelas** merupakan sebuah wadah yang menyimpan properti berupa **atribut dan method**.



Kelas = *Blue Print*

Mengenal Kelas dan Objek

Jika kelas **dieksekusi**, maka kelas tersebut akan terwujud menjadi sebuah **objek**.







Kelas merupakan rancangan atau desain atau blue print dan objek adalah produknya

Objek = *Produk*

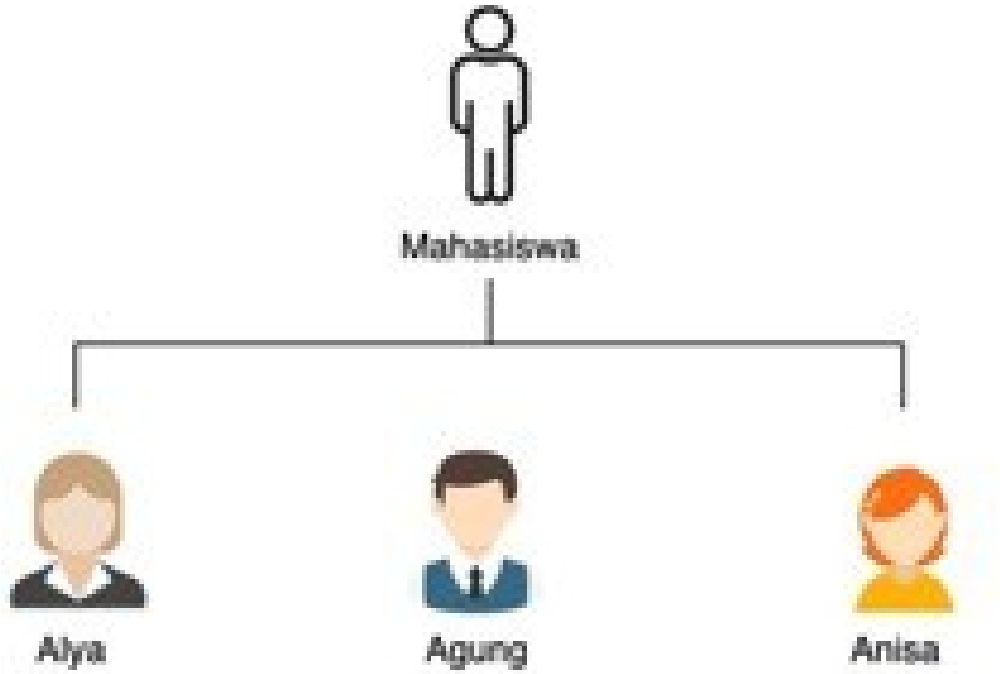
Mengenal Kelas dan Objek pada Python

- Kelas dapat ditulis dalam sebuah modul py atau diidentifikasi dengan keyword **class**.
- Untuk dapat digunakan, kelas harus memiliki **konstruktor** (pembangun)
- Konstruktor berupa method keyword **__init__()**.

```
class nama_class:
    def __init__():
        .......
```

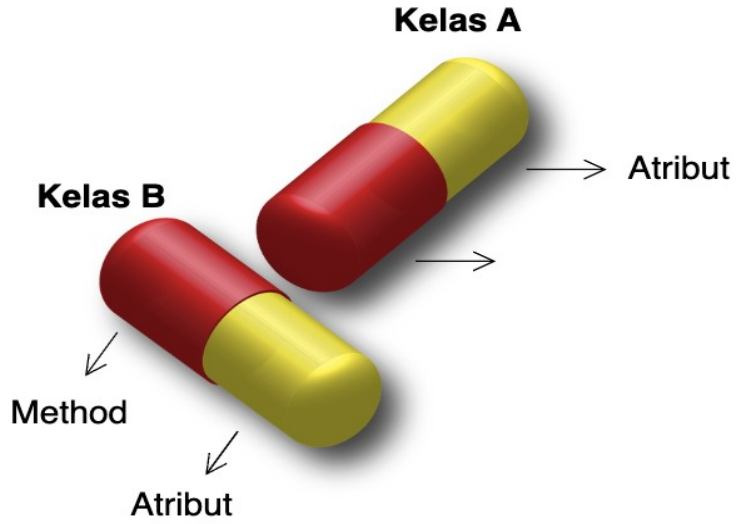
Syntax membuat kelas

Mengenal Kelas dan Objek pada Python



Mengenal enkapsulasi

- Kelas memiliki properti yang berbentuk atribut dan method.
- Jika sebuah sistem program komputer memiliki beberapa kelas, maka kelas bertugas untuk **membungkus atribut dan method** yang identik.
- Pembungkusan atribut dan method ini disebut dengan *encapsulation* (enkapsulasi)

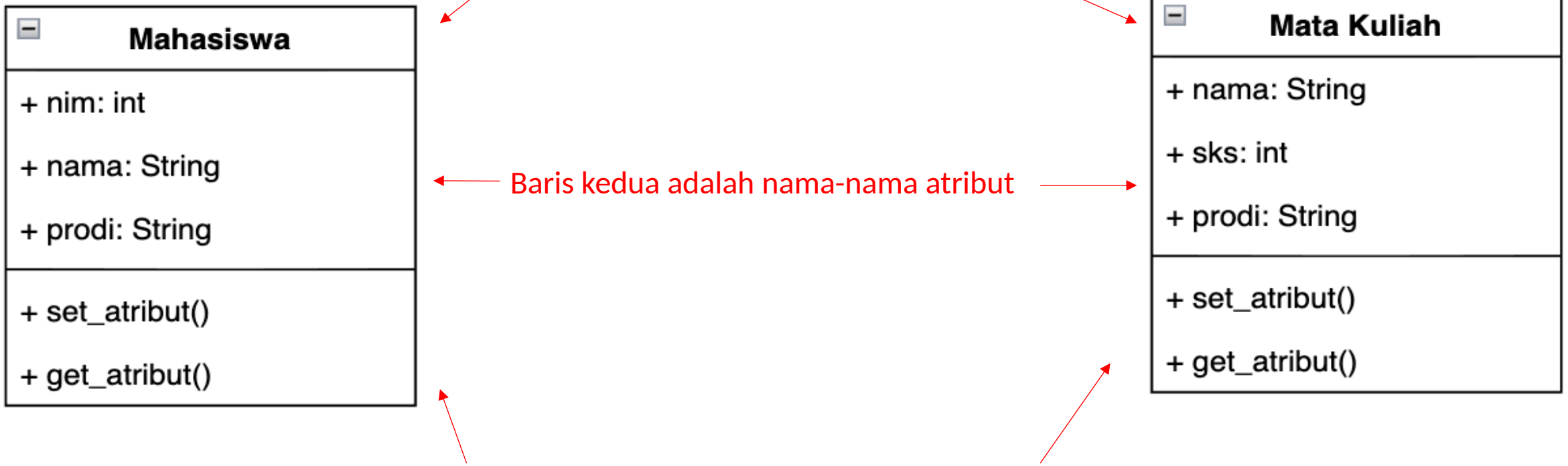


Perancangan Kelas dalam Diagram Kelas

- Salah satu diagram rancangan yang sering digunakan untuk pemrograman berbasis objek adalah **diagram kel**as.
- Diagram kelas secara jelas menampilkan nama kelas, atribut, method, dan juga relasi antar kelas.
- Diagram kelas digambarkan dalam bentuk card yang terbagi menjadi tiga baris.
  - Baris pertama adalah nama kelas
  - Baris kedua adalah nama-nama atribut
  - Baris terakhir adalah method-method yang tersedia.

Perancangan Kelas dalam Diagram Kelas

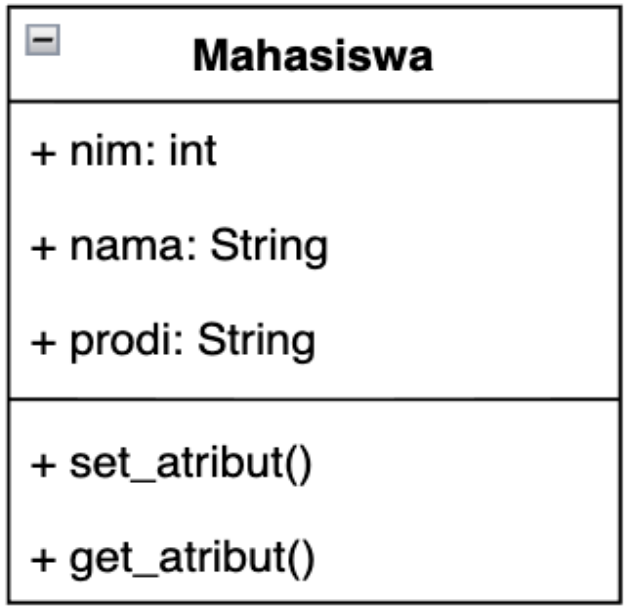
Baris pertama adalah nama kelas



Baris kedua adalah nama-nama atribut

Baris terakhir adalah method-method yang tersedia.

Perancangan Kelas dalam Diagram Kelas



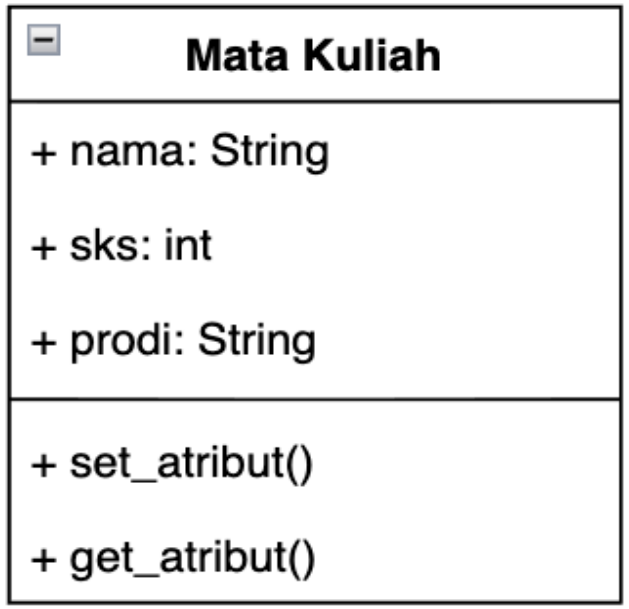

```
class Mahasiswa:

    def __init__(self):
        self.nim=0
        self.nama =''
        self.prodi =''

    def set_atribut(self,val1=0, val2='', val3=''):
        self.nim=val1
        self.nama =val2
        self.prodi =val3

    def get_atribut(self):
        print("Nama \t:",self.nama,
              "\nNIM \t:",self.nim,
              "\nProdi\t:",self.prodi)
```

Perancangan Kelas dalam Diagram Kelas




```
class MataKuliah:

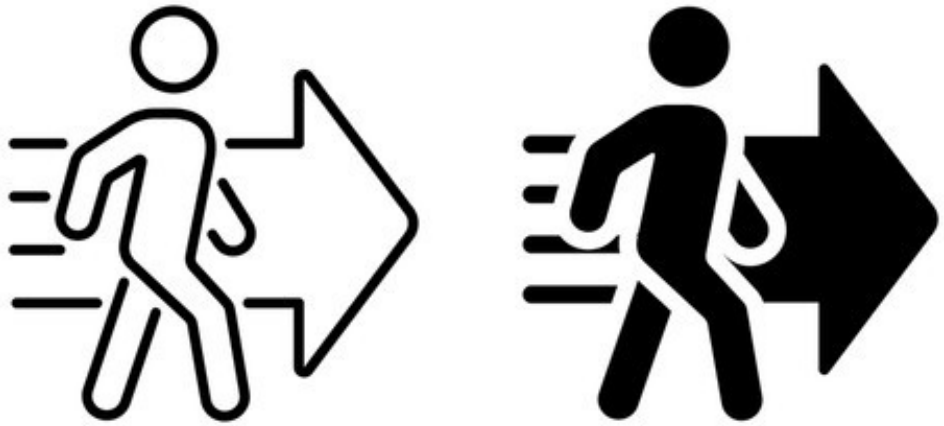
    def __init__(self):
        self.nama =''
        self.sks =0
        self.prodi = ''

    def set_atribut(self,val1='', val2=0, val3=''):
        self.nama =val1
        self.sks =val2
        self.prodi = val3

    def get_atribut(self):
        print("Nama \t:",self.nama,
              "\nsks \t:",self.sks,
              "\nProdi\t:",self.prodi)
```

Mengapa ada keyword *self* ?

- Ketika sebuah objek dibuat dari suatu kelas, setiap objek memiliki data (atribut) yang bisa berbeda.
- Keyword *self* menunjukkan bahwa atribut tersebut adalah milik objek tertentu, bukan milik kelas secara keseluruhan.
- Tanpa *self*, atribut tersebut tidak akan menjadi milik objek.



Praktikum 4.1
Praktikum 4.2
Praktikum 4.3
Praktikum 4.4
Praktikum 4.5

Praktikum 4.1

```
class Mahasiswa:
    pass
```

```
class Mahasiswa:
    def __init__(self):
        pass
```

```
class MataKuliah:
    pass
```

```
class MataKuliah:
    def __init__(self):
        pass
```

Praktikum 4.2

```
mhs1 = Mahasiswa()
mhs2 = Mahasiswa()
mhs3 = Mahasiswa()
```

```
mk1 = MataKuliah()
mk2 = MataKuliah()
mk3 = MataKuliah()
```

```
print(mhs1.__dict__)
print(mhs2.__dict__)
print(mhs3.__dict__)
```

```
print(mk1.__dict__)
print(mk2.__dict__)
print(mk3.__dict__)
```

Praktikum 4.3

```
class Mahasiswa:
    def __init__(self,val):
        self.nama =val

mhs1 = Mahasiswa('Alya')
mhs2 = Mahasiswa('Agung')
mhs3 = Mahasiswa('Anisa')
```

```
print(mhs1.__dict__)
print(mhs2.__dict__)
print(mhs3.__dict__)
```

```
class MataKuliah:
    def __init__(self,val):
        self.nama =val

mk1 = MataKuliah('Dasar Pemrograman')
mk2 = MataKuliah('Pemrograman Objek')
mk3 = MataKuliah('Pemrograman Web')
```

```
print(mk1.__dict__)
print(mk2.__dict__)
print(mk3.__dict__)
```

```
print(mhs1.nama)
print(mhs2.nama)
print(mhs3.nama)
```

```
print(mk1.nama)
print(mk2.nama)
print(mk3.nama)
```

```
class Mahasiswa:
    def __init__(self, val1, val2):
        self.nama =val1
        self.prodi =val2

mhs1 = Mahasiswa('Alya','Informatika')
mhs2 = Mahasiswa('Agung','Elektro')
mhs3 = Mahasiswa('Anisa','Mesin')
```

```
print(mhs1.__dict__)
print(mhs2.__dict__)
print(mhs3.__dict__)
```

```
print('Nama mahasiswa 1:',mhs1.nama)
print('Prodi mahasiswa 1:',mhs1.prodi)

print('Nama mahasiswa 2:',mhs2.nama)
print('Prodi mahasiswa 2:',mhs2.prodi)

print('Nama mahasiswa 3:',mhs3.nama)
print('Prodi mahasiswa 3:',mhs3.prodi)
```

Praktikum 4.3

```
class MataKuliah:
    def __init__(self,val1,val2):
        self.nama =val1
        self.sks =val2
```

```
mk1 = MataKuliah('Dasar Pemrograman',2)
mk2 = MataKuliah('Pemrograman Objek',3)
mk3 = MataKuliah('Pemrograman Web',4)
```

```
print(mk1.__dict__)
print(mk2.__dict__)
print(mk3.__dict__)
```

```
print('Mata Kuliah ',mk1.nama,' memiliki bobot ',mk1.sks,' sks')
print('Mata Kuliah ',mk2.nama,' memiliki ',mk2.sks,' sks')
print('Mata Kuliah ',mk3.nama,' memiliki ',mk3.sks,' sks')
```

```
mhs4=Mahasiswa()
```

```
class Mahasiswa:
    def __init__(self, val1='', val2=''):
        self.nama =val1
        self.prodi =val2

mhs4=Mahasiswa()
mhs4.nama='Andi'
mhs4.prodi='Akuntansi'
print(mhs4.__dict__)
```

```
mhs5=Mahasiswa()
mhs5.nama='Aldo'
print(mhs5.__dict__)
```

```
class Mahasiswa:
    nim=0
    def __init__(self, val1='', val2=''):
        self.nama =val1
        self.prodi =val2

mhs6=Mahasiswa()
mhs6.nama='Ajeng'
mhs6.nim=123
mhs6.prodi='Keuangan'
print(mhs6.__dict__)
```

```
class MataKuliah:
    prodi = ''
    def __init__(self,val1='',val2=0):
        self.nama =val1
        self.sks =val2

mk4 = MataKuliah()
mk4.nama='Pemrograman Objek'
mk4.sks='2'
mk4.prodi='Informatika'
print(mk4.__dict__)
```

Praktikum 4.4

```
class Mahasiswa:
    nim=0
    def __init__(self):
        self.nama =''
        self.prodi =''

    def set_atribut(self,val1='', val2=''):
```

```
        self.nama =val1
        self.prodi =val2

    def get_atribut(self):
        print("Nama \t:",self.nama,
              "\nNIM \t:",self.nim,
              "\nProdi\t:",self.prodi)
```

```
mhs1 = Mahasiswa()
mhs1.set_atribut('Alya','informatika')
mhs1.nim=111
print(mhs1.get_atribut())
```

```
mhs2 = Mahasiswa()
mhs2.set_atribut('Agung','Elektro')
print(mhs2.get_atribut())
```

```
mhs3 = Mahasiswa()
print(mhs3.get_atribut())
```

```
mhs4 = Mahasiswa()
mhs4.nama='Anisa'
mhs4.alamat='Kediri'
mhs4.nim=133
print(mhs4.__dict__)
print(mhs4.get_atribut())
```

```
class MataKuliah:
    prodi = ''
    def __init__(self):
        self.nama =''
        self.sks =0

    def set_atribut(self,val1='', val2=0):
        self.nama =val1
        self.sks =val2

    def get_atribut(self):
        print("Nama \t:",self.nama,
              "\nsks \t:",self.sks)
```

```
mk1 = MataKuliah()
mk1.set_atribut('Dasar Pemrograman','2')
print(mk1.get_atribut())
```

```
mk2 = MataKuliah()
mk2.set_atribut('Pemrograman Objek')
print(mk2.get_atribut())
```

```
mk3 = MataKuliah()
print(mk3.get_atribut())
```

```
mk4.nama='Basis Data'
mk4.sks=3
print(mk4.__dict__)
print(mk4.get_atribut())
```

Praktikum 4.5

```
print(hasattr(mhs1, 'nim'))
print(hasattr(mhs1, 'nama'))
print(hasattr(mhs1, 'prodi'))
print(hasattr(mhs1, 'alamat'))

print(hasattr(mhs4, 'nim'))
print(hasattr(mhs4, 'nama'))
print(hasattr(mhs4, 'prodi'))
print(hasattr(mhs4, 'alamat'))
```

```
print(hasattr(mk1, 'nama'))
print(hasattr(mk1, 'sks'))

print(hasattr(mk4, 'nama'))
print(hasattr(mk4, 'sks'))
```

Mengenal *overloading*

- Python menerapkan konsep overloading dengan memberikan **banyak pilihan** untuk memanggil sebuah method pada kelas

```
class myKelas:

    def __init__(self):
        pass

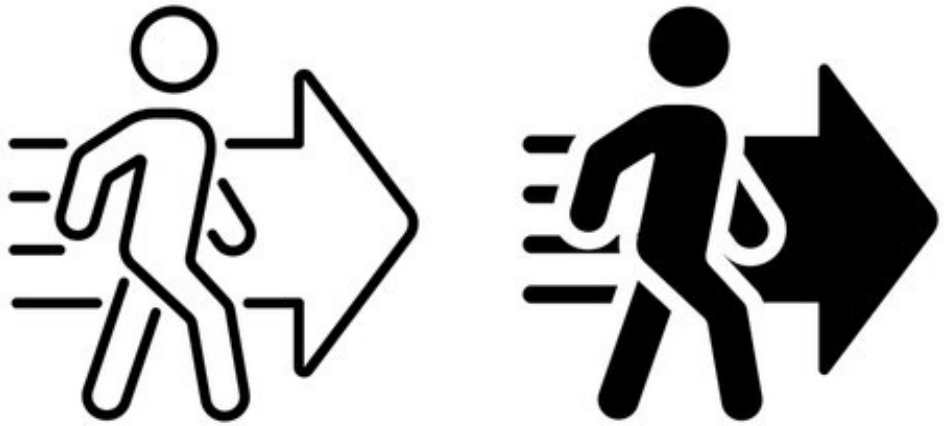
    def myMethod(self,val1='', val2='', val3=''):
        print(val1)
        print(val2)
        print(val3)
```




```
myMethod(val1='Ini value 1')

        myMethod(val2='Ini value 2')

            myMethod(val3='Ini value 3',val1='Ini value 1')
```



Praktikum 4.6

Praktikum 4.6

```
class Mahasiswa:
    nim=0
    def __init__(self):
        self.nama =''
        self.__prodi =''

    def set_atribut(self,val1='', val2=0, val3=''):
        self.nama =val1
        self.nim=val2
        self.__prodi =val3

    def get_atribut(self):
        print("Nama \t:",self.nama,
              "\nNIM \t:",self.nim,
              "\nProdi\t:",self.__prodi)
```

```
mhs5 = Mahasiswa()
mhs5.set_atribut(val1='Furqan',val2=8080123)
mhs5.get_atribut()
```

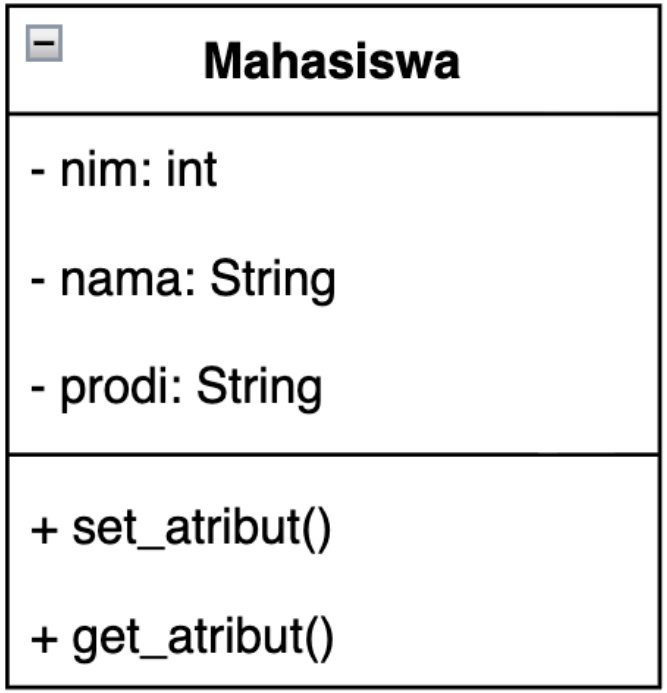
```
mhs6 = Mahasiswa()
mhs6.set_atribut(val1='Farrah',val3='Farmasi')
mhs6.get_atribut()
```

Mengenal akses modifier

- Akses modifier memberikan kita pilihan untuk membatasi property yang terbungkus pada Kelas.
- Akses modifier terdiri dari yakni **public dan private**.
  - Modifier public artinya sebuah properti untuk diakses dimana saja.
  - Modifier private membatasi akses property hanya pada kelas tersebut, sehingga tidak dapat dipanggil dimana saja.
- Dalam diagram kelas, akses modifier private disimbolkan dengan **tanda negatif (-)** sedangkan pada code python disimbolkan dengan **dua garis bawah (__).**

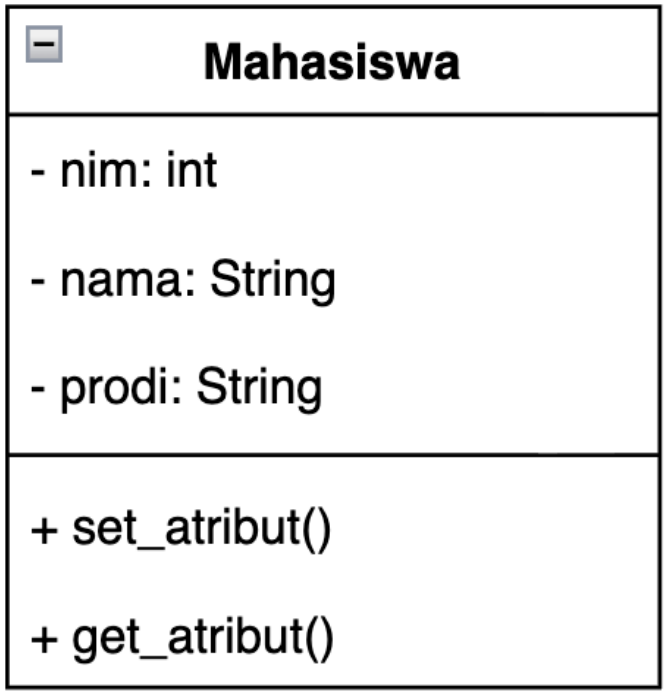
Mengenal akses modifier

Private



Public

Mengenal akses modifier



Private

Public

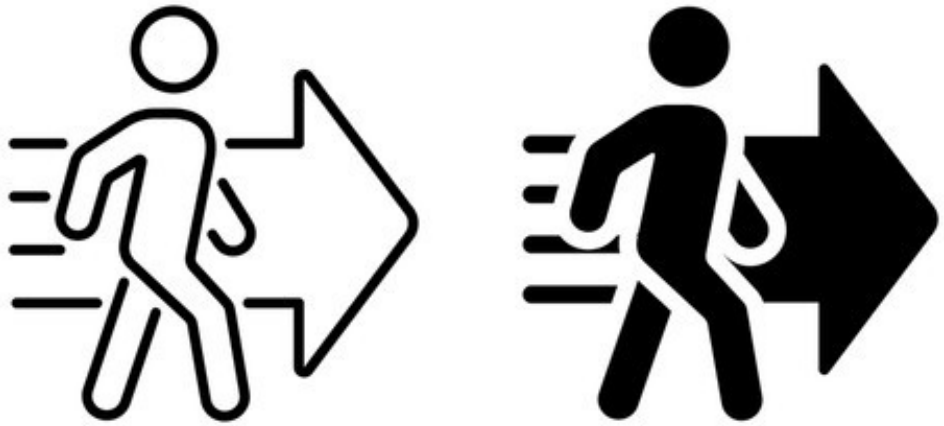
```
class Mahasiswa:
    def __init__(self):
        self.__nama =''
        self.__nim=0
        self.__prodi =''

    def set_atribut(self,val1='', val2=0, val3=''):
        self.__nama =val1
        self.__nim=val2
        self.__prodi =val3

    def get_atribut(self):
        print("Nama \t:",self.__nama,
              "\nNIM \t:",self.__nim,
              "\nProdi\t:",self.__prodi)
```

Perbandingan

Aspek	Public	Private
Penulisan Python	self.nama	self.__nama
Hak akses	Bebas diakses	Hanya dalam kelas
Keamanan data	Rendah	Tinggi
Kemudahan penggunaan	Sangat mudah	Lebih kompleks
Skala penggunaan	Program kecil	Sistem besar dan sensitif



Praktikum 4.7

Praktikum 4.7

```
class Mahasiswa:

    def __init__(self,val1='', val2=0, val3=''):
        self.__nama =val1
        self.nim=val2
        self.__prodi =val3
```

```
mhs1 = Mahasiswa('Alya',1234,'Ilmu Hukum')
print(mhs1.__nama)
```

```
class Mahasiswa:

    def __init__(self,val1='', val2=0, val3=''):
        self.__nama =val1
        self.nim=val2
        self.__prodi =val3

    def get_nama(self):
        return self.__nama

    def get_nim(self):
        return self.nim

    def get_prodi(self):
        return self.__prodi
```

```
mhs1 = Mahasiswa('Alya',1234,'Ilmu Hukum')
print(mhs1.get_nama())
print(mhs1.get_nim())
print(mhs1.get_prodi())
```

Latihan

- Jika terdapat sebuah data tentang kendaraan, maka apa saja properti yang mungkin melekat pada kelas kendaraan tersebut?

- Modelkan rancangan kelas pada no 1 diatas dalam rancangan diagram kelas! Lengkapi pula akses modifier pada masing-masing properti!

Latihan

- Buat kelas AkunBank yang dilengkapi dengan properti:
  a. atribut public: nama
  b. atribut private: __saldo
  c. method public: lihat_saldo() dan setor_uang()

Kemudian tampilkan bagaimana atribut private tidak dapat diakses langsung dari luar kelas.

- Buat kelas Hitung yang memiliki method jumlah() untuk menjumlahkan 2 dan 3 angka dengan menggunakan konsep method overloading!